ACTIA®
ENGINEERING SERVICES

Embedded Team Academy

Advanced C++ training

Prepared and Presented by
Mahdi SAKKA

ACTIA®
ENGINEERING SERVICES

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Agenda

Module 1: C++ Basics

Module 2: C++ Object Oriented Programming

Module 3: C++ Standard Library

Module 4: C++ Object Oriented Programming

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Agenda

Module 1 : C++ Basics

- Introduction to C++
- Variables and types
- Variables Initialization
- Namespaces and scopes
- Functions
- Flow control and error handling
- Exceptions
- Operators and operations on variables
- Constants, typedef and alias
- References and Pointers
- Type conversion
- Memory allocation
- Quiz

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

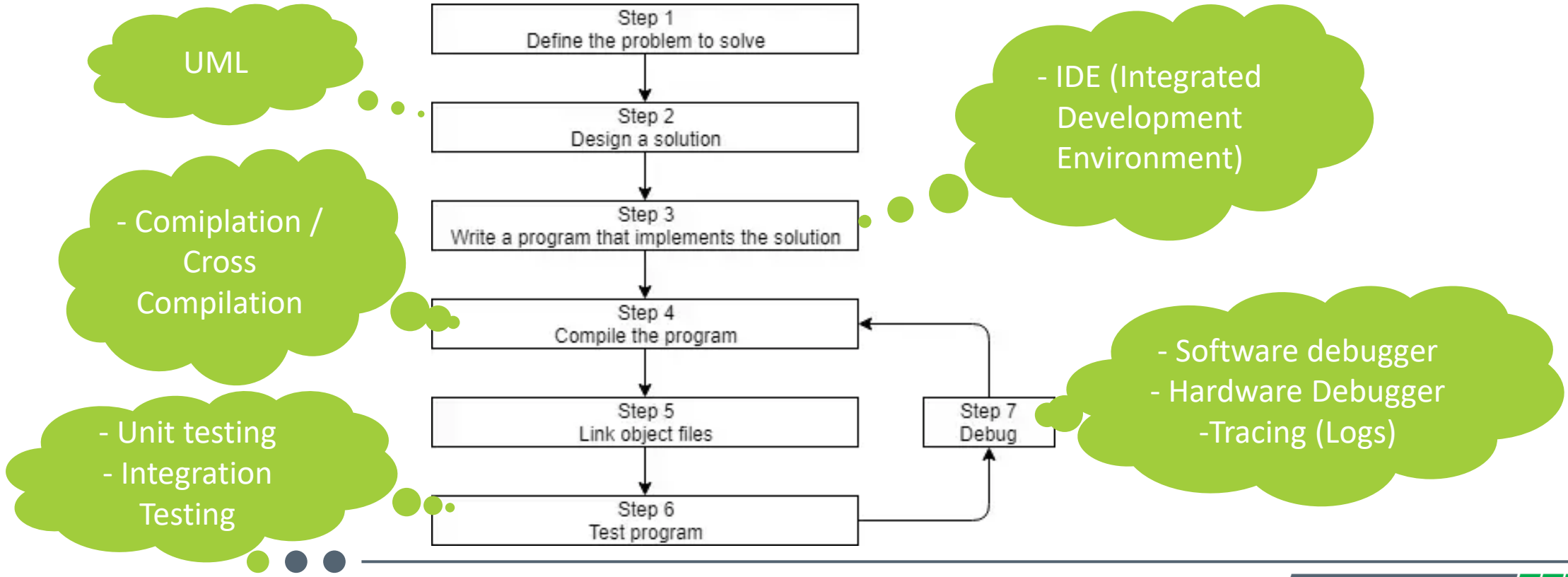
Introduction to C++

- C++ is an Object Oriented Programming Language
  - Extention of C Language
  - Compiled Language
  - Low Level language (no memory management)
  - Trust the programmer
  - Used for high performance systems and embedded systems
  - Benefit from OOP concept

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

Introduction to C++

Software development using C++



Introduction to C++

- C++ program
  - Unique entry point «main»
    - With parameters (console applications)

```
int main(int argc, char * argv[]) {
        return 0;
}
```

    - Without parameters

```
int main() {
        return 0;
}
```

Introduction to C++

- C++ program
  - Header files
    - STL header files without extention
    - Header files «.h» or «.hpp»
  - Source files
    - Files «.cpp» or «.cc»
  - Static libraries
  - Dynamic Libraries

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Introduction to C++

First program

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

```
#include <iostream>

int main(int argc, char* argv[])
{
    for (int i = 0; i < argc; i++) {
        std::cout << "Arg " << i << "= " << argv[i] <<  std::endl;
    }
    return 0;
}
```

Build code (Linux)

```
~$ g++ 01_firstProgram.cpp -o firstProgram
```

Run Binary file

```
~$ ./firstProgram -d opt1 -p opt2
```

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Introduction to C++

Code hierarchy

```
project_name
    |> inc
    |     |> folder1
    |     |      | file1.h
    |     |      | file2.h
    |     |> folder2
    |     |      | file3.h
    |     |      | file4.h
    |> src
    |     |> folder1
    |     |      | file1.cpp
    |     |      | file2.cpp
    |     |> folder2
    |     |      | file3.cpp
    |     |      | file4.cpp
```

Introduction to C++

- Build large projects
  - Use of command line not convini 
  - Use of Make files / CMake files 
- CMake is simpler than Make file

How to build: Create ***build*** directory in workspace

> ~/build$ cmake ..

> ~/build$ make

CMakeLists.txt

```
# Almost all CMake files should start with this
cmake_minimum_required(VERSION 2.8)
# This is your project statement. You should always list
languages;
project(
  ModernCMakeExample
  VERSION 1.0
LANGUAGES CXX)
# Library sample
add_library(MyLibExample simple_lib.cpp simple_lib.hpp)
# Adding something we can run - Output name matches target
name
add_executable(MyExample simple_example.cpp)
# Make sure you link your targets with this command
target_link_libraries(MyExample PRIVATE MyLibExample)
```

Introduction to C++

- Comments in C++
  - Source code should be well commented
  - Comments should be always in ***English***
  - Single line comment C++ style: // this is a single line comment
  - Multi-line comment C Style:
```
/* This is a multi
 * line comment
 */
```
  - Must describe ambigious actions

Variables and types

- A variable is a region of memory that has a value and an identifier (name)
- Instantiation will be at runtime

```
TYPE variable_name = value;
```

- A type will tell the compiler how to deal with the variable
- Should be known at compile time
- Two categories of types:
  - Native C++ types (e.g. char, int, unsigned char, unsigned int, ...)
  - User defined types

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Variables initialization

- Initialization methods:
  - Default:

```
int i;
```

  - Direct

```
int i{0};
int i(0);
```

  - Copy

```
int i = 5;
int i; i = 5;
```



Always initialize variables upon creation

Variables initialization

Literal suffixes: Integer literals

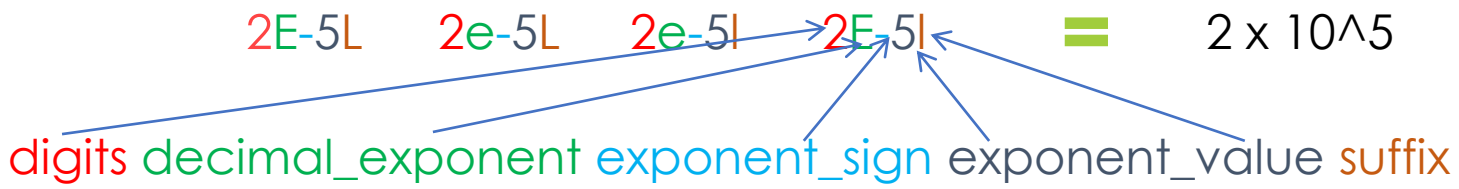
Suffix	Type
(no suffix)	int, long int, long long int
u or U	unsigned int, unsigned long int, unsigned long long int
l or L	long int, long long int, unsigned long int
ul, Ul, uL or UL	unsigned long int, unsigned long long int
ll or LL	long long int

Variables initialization

Literal suffixes: Floating point literals

Suffix	Type
(no suffix)	double
f or F	float
l or L	long double

Exponent



Operations on varibles

- Types of operations
  - Unary: unique operand (e.g. -x)
  - Binary: two operands (e.g. x + y)
  - Ternary (only one conditional operator: condition ? expr1: expr2;)
  - Nullary (only one throw)
- Chaining operators x + y * z → Follow operator priority / use '(' and ')'
- Result of operations vary with operator

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Namespace and scope

- What is a namespace ?

Region where to declare names to remove ambiguities

- Why using namespaces ?

- Remove ambiguities
- Organize source code

- Define a namespace

```
namespace namespace_name {
    // code here
} // namespace namespace_name
```

- Namespace type
  - User defined namespace
  - Global namespace

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Namespace and scope

- Access a name in namespace

```
namespace namespace_name {
    int i{5};
} // namespace namespace_name

namespace_name::i = 10;
```

- «using» directive

```
namespace namespace_name {
    int i{5};
} // namespace namespace_name

using namespace namespace_name;
i = 10;
```

- Special namespace «std::»



Do not used ‘using namespace std;’

Namespace and scope

- Unnamed (anonymous) namespace
  - Used when you have a lot of content that you want to ensure stays local to a given file
- Inline namespace
  - To manage versions of function
  - Just change inline to choose version

```cpp
namespace // unnamed namespace
{
    void doSomething() // can only be accessed in this file
    {
        std::cout << "v1\n";
    }
}

int main()
{
    doSomething(); // we can call doSomething() without a namespace prefix
    return 0;
}
```

```cpp
inline namespace v1 // declare an inline namespace named v1
{
    void doSomething()
    {
        std::cout << "v1\n";
    }
}

namespace v2 // declare a normal namespace named v2
{
    void doSomething()
    {
        std::cout << "v2\n";
    }
}

int main()
{
    v1::doSomething(); // calls the v1 version of doSomething()
    v2::doSomething(); // calls the v2 version of doSomething()
    doSomething(); // calls the inline version of doSomething() (which is v1)
    return 0;
}
```

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Namespace and scope

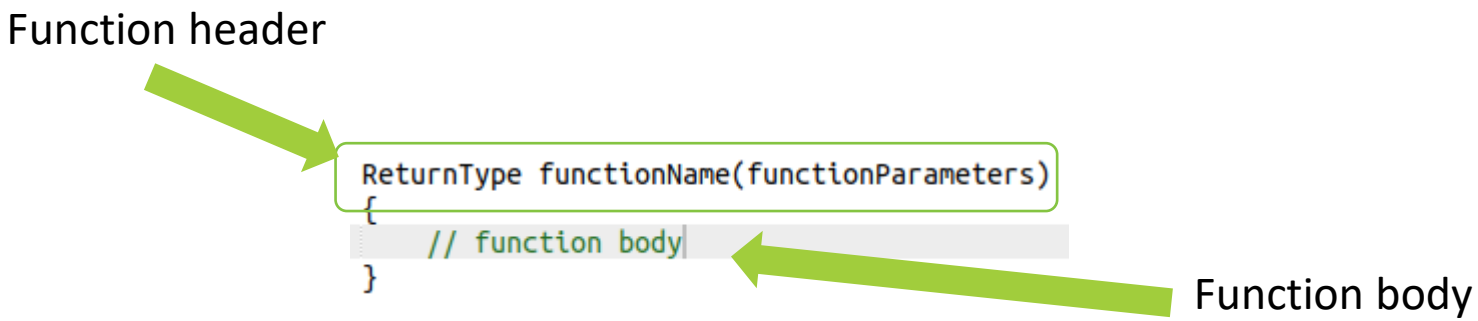
- Scope determines when an identifier can be seen and used

```
{
   int i{0};
}
// i is no longer available here (out of scope)

int add(int x, int y) {
   return x + y;
}
// x and y are no longer available here (out of
scope)
```

Functions

- Reusable sequence of statements to do particular job
  - Organize source code
  - Increase readability
  - Facilitate maintenance of code



Functions

- Function can call other functions

```
#include <iostream>

void doB()
{
    std::cout << "In doB()\n";
}
void doA()
{
    std::cout << "Starting doA()\n";
    doB();
    std::cout << "Ending doA()\n";
}

// Definition of function main()
int main()
{
    std::cout << "Starting main()\n";
    doA();
    std::cout << "Ending main()\n";

    return 0;
}
```



```
Starting main()
Starting doA()
In doB()
Ending doA()
Ending main()
```

- Nested functions are not legal in C

```
int main()
{
    void foo() // Illegal: this function is nested inside function main()
    {
        std::cout << "foo!" << std::endl;
    }

    foo(); // function call to foo()
    return 0;
}
```

Functions

- Funtions return value
  - Not returning value (void)
  - Returning a type (int, char, float, ...)

```
int foo()
{
    return 0;
}
```

return literal value

```
int foo()
{
    int x{0};

    // processing here

    return x;
}
```

return variable value

```
void foo()
{
    if (condition)
    {
        return;
    }
    // processing here
}
```

no return value

- Returning function should always have a return value in every exit point, otherwise we will have an undefined behavior

Functions

- Function arguments: need to pass data to and from functions:
  - Pass arguments by value
  - Pass arguments by address
  - Pass arguments by reference

```
int addFive(int x)
{
    return x + 5;
}
```

argument passed by value

```
bool performOperation(int x)
{
    if (x > 0)
    {
        // change x value ???
        return true;
    }
    else
    {
        return false;
    }
}
```

argument passed by value but need to change an argument value



Functions

- Special functions: Lambdas
  - Anonymous functions that can be created inside another function (nesting function)

```
[ captureClause ] ( parameters ) -> returnType
{
    statements;
}
```

- The capture clause can be empty if no captures are needed.
- The parameter list can be empty if no parameters are required.
- The return type is optional, and if omitted, auto will be assumed

  - Can be assigned to a variable (give a name to lambda)
  - A lambda call creates a new instance of that lambda

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Functions

- Special functions: Lambdas
  - Generic lambda:

```
auto lambda = [](auto i) -> auto {
   return true;
};
```

  - Return type deduction: must have same return type as the first return sentence

```
auto divide{ [](int x, int y, bool intDivision) { // note: no specified return type
    if (intDivision)
        return x / y; // return type is int
    else
        return static_cast<double>(x) / y; // ERROR: return type doesn't match previous return type
}};
```


ACTIA®

Functions

- Special functions: Lambdas
  - Captures allow using variables from surrounding scope
  - Capture types:
    - By constant value: [=] or [var1, var2, .. varn]



    - By mutable value [=] () **mutable** {}
    - By reference or address [this] or [&] or [&var1, **pointer** ... &varn]



☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Functions

Special functions: Lambdas



Be careful ! Dangling passed variables by reference

```
// returns a lambda
auto printHello (const std::string & hello)
{
     // Capture name by reference and return the lambda.
     return[&]() {
          std::cout << "Say " << hello << '\n';    // Undefined behavior
     };
}
```

```
// sayHello is the lambda returned by printHello.
auto sayHello { printHello("Hello")};

// Call the lambda function
sayHello ();
```

Copy of lambda

```
void myInvoke(const std::function<void()>& fn){
    fn();
}

int main(){
    int i{ 0 };
    // Increments and prints its local copy of @i.
    auto count{ [i]() mutable {  std::cout << ++i << '\n'; } };

    myInvoke(std::ref(count));
    myInvoke(std::ref(count));
    myInvoke(std::ref(count));

    return 0;
}
```

Flow control and error handling

- Conditional statement
  - «**if**» statement:

```
if (boolean_condition) {
    // true_statement(s)
}
else {
    // false_statement(s)
}
```

```
int i{0};

if (i == 0) {
    std::cout << "Condition is verified" << std::endl;
}
else {
    std::cout << "Condition is not verified" << std::endl;
}
```

```
int i{0};
int j{0};

if (i != 0) {
    std::cout << "Condition is verified" << std::endl;
}
else if (j == 0) {
    std::cout << "Second condition is verified" << std::endl;
}
else {
    std::cout << "None of conditions is verified" << std::endl;
}
```

Flow control and error handling

- Conditional statement
  - «**switch**» statement

```
switch (value) {
   case value1:
            // processing here if value equals value1
            break; (return;)
   case value2:
            // processing here if value equals value2
            break; (return;)
   default:
            // default processing here
            break; (return;)
}
```

```
int i{0};

switch (i) {
    case 0:
        std::cout << "i eqauls 0" << std::endl;
        break;
    case 1:
        std::cout << "i eqauls 1" << std::endl;
        break;
    default:
        std::cout << "i different from 0 and 1" << std::endl;
        break;
}
```

Flow control and error handling

Conditional statement

«**conditonal operator**» statement

condition ? true_statement : false_statement;

```
int i{0};

i == 0 ? (std::cout << "i equals 0" << std::endl) : (std::cout << "i different from 0" << std::endl);
```

Flow control and error handling

Looping statement

«**while**» statement

```
while (boolean_condition) {
    // code to be repeated
    [break;]
    [continue;]
}
```



«boolean_condition» must be true to enter the loop

```
int i{0};

while (i == 0) {
    std::cout << "While loop" << std::endl;
    i = 1;
}
```

Flow control and error handling

Looping statement

«**do .. while**» statement

```
do {
    // code to be repeated
    [break;]
    [continue;]

} while (boolean_condition) ;
```



«boolean_condition» can be not true to enter the loop

```
int i{0};
do {
    std::cout << "Do .. While loop" << std::endl;
    i = 1;
}
while (i == 0);
```

Flow control and error handling

Looping statement

«**for**» statement

```
for(init_statement; boolean_condition; expression) {
    // code to be repeated
    [break;]
    [continue;]
}
```



«init_statement» is executed before enter the loop
«boolean_condition» must be not true to enter the loop
«expression» is executed each time after code to be repeated

```
for (int i = 0; i < 10; i = i + 2) {
    std::cout << "For loop with counter " << i << std::endl;
}
```

Flow control and error handling

- Exercice
  - Write a function that displays the elements of a vector in separate lines using std::cout and use lambdas to multiply elements in even index by 2 and odd index by 3. Use the same processing with for, do..while and while.

```
std::vector<int> vect{3, 40, 10, 56, 2, 4};

int vect[]{3, 40, 10, 56, 2, 4};
```

Flow control and error handling

- Error handling strategies
  - Handle error within the function
  - Return error to the caller
  - Throw Exception
  - Halt the program

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Flow control and error handling

- Assertion
  - Used to verify a condition that is essential to program workflow
  - Used in runtime
  - If condition is not true, program will terminate

```cpp
#include <cassert> // for assert()

void performOperation(int x)
{
    assert(x > 0);
    // function implementation
}
```

- Static assertion
  - Used to verify a condition
  - Used at compile time
  - If condition is not true, compilation error

```cpp
void performOperation(int x)
{
    static_assert(sizeof(x) == 4, "We cannot handle int with size <> 4");
    // function implementation
}
```

Exceptions

Why Exceptions ?



```cpp
bool checkParameter(const std::string& name) {
    // implementation here
    // return check result
}

int main()
{
    if (checkParameter("Param1") != true) {
        return ERROR;
    }
    if (checkParameter("Param2") != true) {
        return ERROR;
    }
    if (checkParameter("Param3") != true) {
        return ERROR;
    }

    return 0;
}
```

```cpp
double divide(double x, double y) {
    if (y == 0) {
        // error !!
        // how to handle
    }
    else {
        return (x / y);
    }
}
```



```cpp
double divide(double x, double y, bool& outSuccess)
{
    if (y == 0)
    {
        outSuccess = false;
        return 0.0; // wrong return
    }

    outSuccess = true;
    return (x / y);
}
```



Exceptions to decouple handling errors from typical code flow

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Exceptions

- To trigger exception «**throw**»
- To handle exception

«**try ... catch ...**»

```
throw -1; // throw a literal value
throw ENUM_INVALID; // throw an enum value
throw "Can't divide by 0"; // throw a literal C-style (const char*) string
throw dX; // throw a variable that was previously defined
throw MyException("Fatal Error"); // Throw an object of class MyException
```

```
try
{
    // Statements that may throw exceptions you want to handle here
}
catch (exception_simple_type1 x)
{
    // Any exceptions of type exception_simple_type1 thrown within the above try block
    std::cerr << "We caught an exception_simple_type1 exception with value: " << x << std::endl;
}
catch (exception_simple_type2) // no variable name since we don't use the exception itself in the catch block below
{
    // Any exceptions of type double thrown within the above try block
    std::cerr << "We caught an exception of type exception_simple_type2" << std::endl;
}
catch (const class_name1&) // catch classes by const reference
{
    // Any exceptions of type class_name thrown within the above try block
    std::cerr << "We caught an exception of type std::string" << std::endl;
}
catch (const class_name2& name) // catch classes by const reference
{
    // Any exceptions of type class_name2 thrown within the above try block
    std::cerr << "We caught an exception of type class_name2 " << name << std::endl;
}
catch (...)
{
    // Any exceptions of a type not in the list
    std::cerr << "We caught an exception of non handled type" << std::endl;
}
```

Exceptions

- Exception from called function

```
void foo() {
    throw "This is an exception !";
}

int main()
{
    try {
        foo();
    }
    catch(const char* e)
    {
        std::cout << "Caught exception: " << e << std::endl;
    }

    return 0;
}
```

- Stack unwinding: may not happen if no exception handler is found (something to avoid)

Exceptions

- Exception dangers and downside:
  - Cleaning up resources: be sure to clean allocated data and resources

```
int * ptr{nullptr};

try
{
    ptr = new int;
    throw 1;
}
catch (...)
{
    std::cerr << "Exception caught";
}

if (ptr != nullptr) {
    delete ptr;
}
```

```
try
{
    openFile();
    writeData();
}
catch (...)
{
    std::cerr << "Exception caught";
}

closeFile();
```

  - Performance price to pay when exception is thrown

Exceptions

- «**noexcept**» specifier
  - Used to specify that given function doesn't throw exception
  - Exception can be handled locally in noexcept function
  - std::terminate is called is the function throws exception
- «noexcept» with boolean
  - noexcept(true) equivalent to noexcept means not throwing
  - noexcept(false) means potentielly throwing

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Exceptions

- When to use «noexcept»
  - Specify that function can be called safely without try catch block
  - Compiler can optimize more noexcept functions

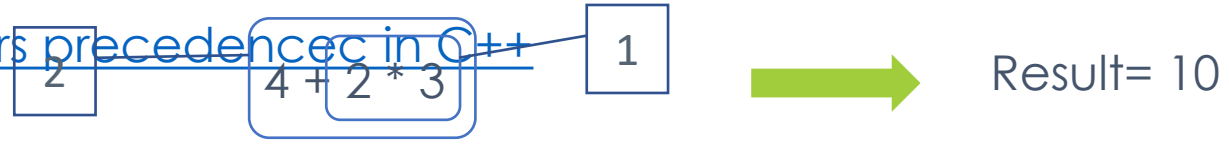
- Optional hint in function declaration
  - «***void foo() throw()***» function doesn't throw exception
  - «***void foo() throw(int, std::runtime_error)***» may throw one of two types mentioned
  - «***void foo() throw(...)***» may throw any type

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Operators and operations on variables

- C++ expression is treated by compiler according to operators precedence
- Link to operators precedencec in C++



- Operators in higher rows are bound tighter to its argument than another in a lower row
- Unary prefix operators (++a) always associate right-to-left
- Unary postfix operators (a++) always associate left-to-right



Best practice is to use parentheses to avoid ambiguities

Precedence	Operator	Associativity
1	::	Left-to-right →
2	a++ a-- *type*() *type*{} a() a[] . ->	
3	++a --a +a -a ! ~ (*type*) *a &a sizeof co_await new new[] delete delete[]	Right-to-left ←
4	.* ->*	Left-to-right →
5	a*b a/b a%b	
6	a+b a-b	
7	<< >>	
8	<=>	
9	< <= > >=	
10	== !=	
11	a&b	
12	^	
13	|	
14	&&	
15	||	
16	a?b:c throw co_yield = += -= *= /= %= <<= >>= &= ^= |=	Right-to-left ←
17	,	Left-to-right →

Constants, typedef and alias

- Constant is a value that may not be changed
  - Constant variable

```
const double gravity { 9.8 };  // preferred use of const before type
int const sidesInSquare { 4 }; // okay but not preferred
```

const keyword

  - Constant function (should be on class member function)
  - Constant return

```
const int fn()
{
    const int i{5};

    return i;
}
```

return **const** variable

  - Constant expression: all values must be known at compile time

Constants, typedef and alias

- Compile time constants

```
const int z { 3 };     // 3 is a constant expression, so z is compile-time const
const int a { 1 + 2 }; // 1 + 2 is a constant expression, so a is compile-time const
const int b { z * 2 }; // z * 2 is a constant expression, so b is compile-time const
```

  - Allow compiler to make more and more optimisations

- Runtime constants

```
int fn()
{
     int i{5};
     return i;
}

const int z { fn() };     // z is run-time const
```

- «***constexpr***» keyword: necessarly a compile time constant

```
constexpr double gravity { 9.8 }; // ok: 9.8 is a constant expression
constexpr int sum { 4 + 5 };      // ok: 4 + 5 is a constant expression
constexpr int something { sum };  // ok: sum is a constant expression

int age{};
constexpr int myAge { age };      // compile error: age is not a constant expression
constexpr int f { five() };       // compile error: return value of five() is not a constant expression
```

Constants, typedef and alias

- Constexpr **functions**:
  - Can be evaluated either at runtime or compile time

```
constexpr int g { greater(5, 6) };            // case 1: evaluated at compile-time
std::cout << g << " is greater!\n";

int x{ 5 }; // not constexpr
std::cout << greater(x, 6) << " is greater!\n"; // case 2: evaluated at runtime

std::cout << greater(5, 6) << " is greater!\n"; // case 3: may be evaluated at either runtime or compile-time
```

  - Must satisfy many conditions **to be evaluated at compile time**: Refer to documentatio

```
constexpr int factorial(int n)
{
    return n <= 1 ? 1 : (n * factorial(n - 1));
}
```

Constants, typedef and alias

- Inline **functions**:
  - Used to try to indicate to compiler that this is an inline functior
  - Compiler can accept inlining or refuse inlining and process it as simple function
  - Should have condition to accept inlining the function
  - If inline is accepted, function will not be called as a function but code will be replaced

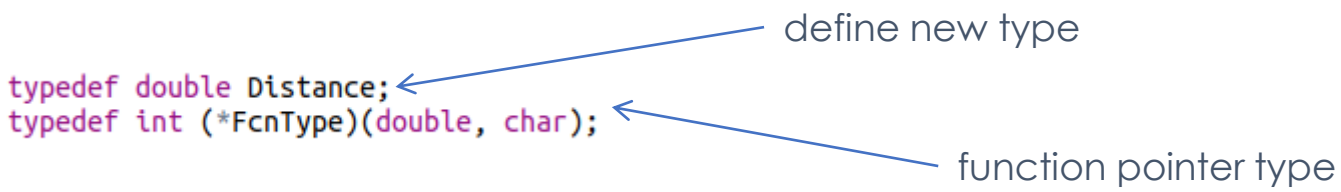
```
inline int max(int x, int x)
{
    return (x > y) ? x : y;
}
```

=> gain of execution time

  - Inline conditions, see link
  - A function declared constexpr is implicitly an inline function.

Constants, typedef and alias

- Typedef: old way to define new types (kept for compatibility)



- The use of **typedef** is deprecated

→ use alias instead

Constants, typedef and alias

- Alias: keyword «**using**»: creates an alias for an **exiting type**

```
using OurDouble = double;

OurDouble value{2.0};
```

- Can be declared at any emplacement of code
- Alias are not type saf

```
using Distance = double;
using Speed = double;

Distance remainDistance { 90.0 };
raiseCarSpeed( remainDistance );
```

compiler process both types as double, do not make difference between types (can mix types)

- Why use alias in source code ?
  - Make complex type easier

```
using SimpleVector = std::vector< std::vector< std::pair<std::string, int> > >;
```

  - Make code easier to read

```
using Distance = double;

Distance getRemainingDistance();
```

  - Make code maintenance easier

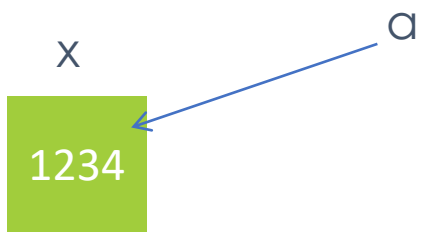
Provisoire Public Interne Confidentiel Secret

References and pointers

- rvalue vs. lvalue
  - lvalue (left value)
    - is an expression that evaluates to identifiable object or function
    - can be modified (unless constant)
  - rvalue (right value)
    - is an expression that is not a lvalue, generally a literal or a return of called function
    - cannot be modified

References and pointers

- References:
  - A reference is an **alias** to an **existing object**
  - Identical to the object being referenced




```
int x{1234};
int& a = x;
```

  - Two types:
    - lvalue reference
    - rvalue reference

References and pointers

- References: lvalue references

```
int& invalidRef1;   // error: references must be initialized

int x { 5 };
int& ref { x }; // valid: lvalue reference bound to a modifiable lvalue

const int y { 5 };

int& invalidRef2 { y };  // invalid: can't bind to a non-modifiable lvalue
int& invalidRef3 { 0 }; // invalid: can't bind to an r-value

const int& ref2 { 0 }; // valid but cannot be modified

int z { 3 };
const int& ref3 { z }; // valid but cannot be modified
```

- It must have same type as value being referenced
- No reference to void type

References and pointers

- References: lvalue references retuned in functions

compile error:
reference to local variable

```
int x{0};

int& fun1()
{
    return x;
}

int& fun2()
{
    int y {0};
    return y;   // Compile error
}

int& fun3()
{
    static int z{0};
    return z;
}

int main()
{
    int a { fun1() };
    int b { fun2() };
    int c { fun3() };
    return 0;
}
```

References and pointers

- References: Pass value by reference

  Call to fun() will make a **copy** of argument:
  expencive in execution time

```
void fun(std::string str)
{
}
```

  Call to fun() won't make a copy
  * **ADVANTAGE1**: gain of time
  * **ADVANTAGE2:** modify argument in function will modify original value

```
void fun(std::string& str)
{
}
```

- Pass by const reference

  Call to fun() won't make a copy
  * **ADVANTAGE1**: gain of time
  * **ADVANTAGE2:** do not modify original value
  * **ADVANTAGE3:** can be bound to lvalue, const lvalue and rvalue

```
void fun(const std::string& str)
{
    str = "New value"; // not allowed
}
```

References and pointers

- References: rvalue references

```
int&& rRef { 20 };
rRef = 40;
// int &&ref = x; not allowed
```

- Use of rvalue references,
  - Working with move contructor and move assignment (will be seen later)
  - Cannot bind non-const lvalue reference of type ‘int&‘ to an rvalue of type ‘int’ (see provided sample code)
  - Cannot bind rvalue references of type ‘int&&‘ to lvalue of type ‘int’ (see provided sample code)

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

References and pointers

- Pointers: special object type that contains address of other object

Allocate and free pointer to one element

Allocate and free pointer to many elements

```
int x{100};
int* ptr{&x}; // pointer to x

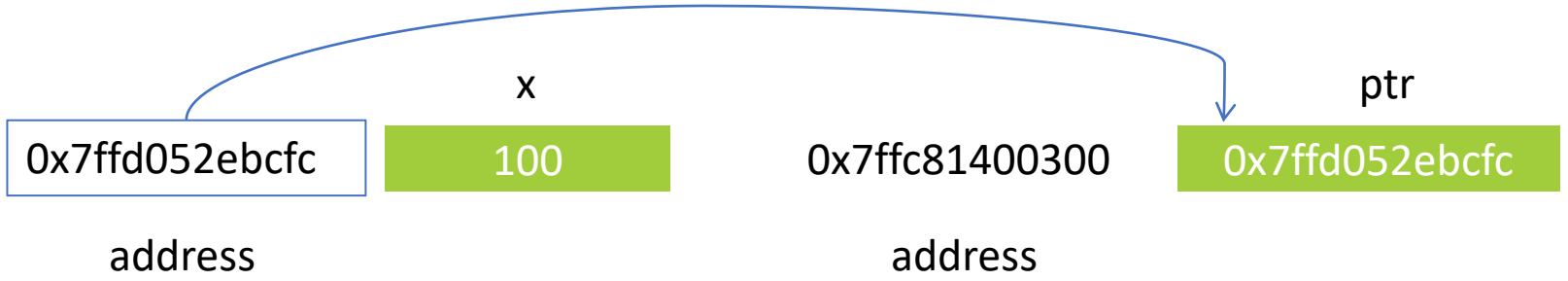
int y = *ptr; // get content of pointer

int* ptr1 = new int;
delete ptr1;

int* ptr2 = new int[10];
delete[] ptr2;
```

Address operator '&'

Dereferencing operator '*'



References and pointers

- Pointers:
  - size of pointer depends on address bus size
    - on 32-bit machine -> size of pointer 32bits (4 Bytes)
    - on 64-bit machine -> size of pointer 64bits (8 Bytes)



- Dangling pointers:
  - * Holding an address of a no longer exist
    variable or object
  - * Pointing to an illegal address and trying
    access it (e.g. 0x0)

```
int x{ 5 };
int* ptr{ &x };
std::cout << *ptr << '\n'; // valid
{
    int y{ 6 };
    ptr = &y;
    std::cout << *ptr << '\n'; // valid
} // y goes out of scope, and ptr is now dangling
std::cout << *ptr << '\n'; // undefined behavior
ptr = 0x0;
std::cout << *ptr << '\n'; // undefined behavior
```

References and pointers

- Pointers: best practices:
  - Always initialize the pointer when declared (nullptr, allocate or pointing to variable)
  - Always check if it is a null pointer before use
  - Promote the use of lvalue references than pointers when possible
  - Always check that allocated pointers are freed at every exit point

References and pointers

- Pointers and const keyword

```
int z{ 5 };
const int x { 5 }; // x is now const
int* ptr { &x };   // compile error: cannot convert from const int* to int*
/// Pointer to const
const int y{ 5 };
const int* ptrc { &y }; // okay: ptr is pointing to a "const int"
*ptrc = 6; // not allowed: we can't change a const value
ptrc = &x; // we can change since pointer is not const (but y was const)
/// Constant pointer
int* const cptr { &z }; // okay: the const pointer is initialized to the address of z
cptr = &y; // error: once initialized, a const pointer can not be changed.
*cptr = 6; // okay value is changed and not the pointer
const int* const ptr { &x }; // a const pointer to a const value
```

- Different types
  - Pointer to const
  - Const Pointer
  - Const Pointer to Const

References and pointers

- Pointers: pass by address

Call to fun() will make a **copy** of argument:
expencive in execution time

```
void fun(std::string str)
{
}
```

Call to fun() won't make a copy

```
void fun(std::string* str)
{
}
```

* **ADVANTAGE1**: gain of time
* **ADVANTAGE2:** modify argument in function will modify original value

References and pointers

- Return by reference:

Make a copy of returned value: expensive in memory and exec time

```
std::string fun()
{
    std::string str{"ABC....."};
    return str;
}
```

The returned variable by reference should live as long as the returned value is needed

```
std::string& fun1()
{
    static std::string str{"ABC....."};
    return str;
}
const std::string& fun2()
{
    static std::string str{"ABC....."};
    return str;
}
```

References and pointers

- Return by address:
  - Almost same as return by reference
  - Advantage: can return ***nullptr*** when empty return is needed

The returned variable by address should point to a valid variable

```
std::string* fun1()
{
    static std::string str{"ABC....."};
    return &str;
}
```

References and pointers

- Exercice
  - Write a function to perform swap of two integers using pointers and anoter one using references.

swap(a, b)

Type conversion

- Type conversion is to convert from one type to another
  - Implicit conversion when compiler convert type when mismatch detected
  - Explicit conversion is mentioned explicitly in code

```
void print(double x)
{
    std::cout << x;
}

int main()
{
    int y {5};
    print(y); // y is of type int

    return 0;
}
```

Implicit conversion from int to double

```
void print(double x)
{
    std::cout << x;
}

int main()
{
    int y {5};
    print(static_cast<double>(y));

    return 0;
}
```

Explicit conversion from int to double using static_cast

Type conversion

Implicit type conversion

```
double d{3}; // int value 3 implicitly converted to type double
d = 6; // int value 6 implicitly converted to type double

float doSomething()
{
    return 3.0; // double value 3.0 implicitly converted to type float
}

double division{ 4.0 / 3 }; // int value 3 implicitly converted to type double

if (5) // int value 5 implicitly converted to type bool
{
}

void doSomething(long l)
{
}

doSomething(3); // int value 3 implicitly converted to type long
```

Type conversion

- Implicit type conversion
  - Programmer is responsible of casting
  - Promotion conversion is allowed (from narrow type to wider type)
  - Narrowing conversion is risky
    - Loose precision (double to float)
    - Wrong value (such as int to char)

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Type conversion

- Explicit type conversion : force the conversion type
  - C-Style cast (must not be used in modern C++)
  - Using casting operators
    - static_cast
    - dynamic_cast
    - const_cast
    - reinterprete_cast

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Type conversion

- Explicit type conversion : static_cast
  - Used to convert one type to another
  - Conversion during compile time (more safe)

```
int a = static_cast<int>('R');
```

Type conversion

- Explicit type conversion : dynamic_cast
  - Used to convert mother class to child classes
  - Will be seen later after object oriented module

```
Child * a = dynamic_cast<Child*>(parent*);
```

Provisoire Public Interne Confidentiel Secret

ACTIA®

Type conversion

- Explicit type conversion : const_cast
  - Used to cast away contness of variables
  - Used to change non-const class members inside a const member function
  - Remove and modify constness of pointer that points to non-constant value

```
T a = const_cast<T>(const T);
```

Type conversion

- Explicit type conversion : reinterprete_cast
  - Used to convert a pointer of a type to another pointer of another type
  - Compiler doesn't perform any check
  - Be careful on using it

```
T* a = reinterprete_cast<T*>(U*);
```

Memory Allocation

- C++ supports three memory allocation types:
  - Static memory allocation: happens for static and global variables.
  - Automatic memory allocation: happens for function parameters and local variables.
    - → Size must be known at compile time
    - → Allocation and free will be done automatically at instantiation / destruction
  - Dynamic memory allocation: happens on user demand to allocate and free memory
    - → Size of memory can be determined at runtime

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Memory Allocation

- Dynamic memory allocation:
  - Allocation
    - single variable → **type_name*** ptr_var { new **type_name{**init_value**} }**;
    - arrays → **type_name*** ptr_var { new **type_name[]{**init_value1, init_value2, **} }**;
  - Free
    - single variable → delete ptr_var; ptr_var = nullptr;
    - arrays → delete[] ptr_var; ptr_var = nullptr;



Always be sure to free allocated memory at every exit point (function, scope of pointer ...)

Memory Allocation

- Dynamic memory allocation:



Dangling pointer: do not use pointer after being freed (may cause crash)

```
int* ptr{ new int }; // dynamically allocate an integer
*ptr = 7; // put a value in that memory location

delete ptr; // return the memory to the operating system.  ptr is now a dangling pointer.

std::cout << *ptr; // Indirection through a dangling pointer will cause undefined behavior

*ptr = 7; // put a value in that memory location will also lead to undefined behavior.
delete ptr; // trying to deallocate the memory again will also lead to undefined behavior.
```



Best practice is to set pointer to ‘nullptr’ right after delete



Do not access an pointer value that overflow the allocated size (may cause crash)

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

Memory Allocation

- Dynamic memory allocation:
  - «new» operator call may fail in rare cases (e.g. no memory space left)



Best practice is to check if allocated pointer value is not «nullptr»

  - Do not use an already allocated pointer to allocate it again



We need sometimes to check if pointer is «nullptr» before allocation

Quiz

- C++

ACTIA®
ENGINEERING SERVICES

AN ENTREPRENEURIAL SPIRIT

TO MOVE FORWARD TOGETHER

ACTIA Engineering Services

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®